João Paulo Garcia, João Pedro Magalhães, Miguel Luis Schwengber

Aplicação de Protocolos de Oblivious Transfer para Preservação de Privacidade em Transações Digitais

São Paulo, SP
2026

João Paulo Garcia, João Pedro Magalhães, Miguel Luis Schwengber

Aplicação de Protocolos de Oblivious Transfer para Preservação de Privacidade em Transações Digitais

Trabalho de conclusão de curso apresentado ao Departamento de Engenharia de Computação e Sistemas Digitais da Escola Politécnica da Universidade de São Paulo para obtenção do Título de Engenheiro.

Universidade de São Paulo – USP
Escola Politécnica
Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Orientador: Prof. Dr. Thales Areco Bandiera Paiva

São Paulo, SP
2026

Sumário

1 INTRODUÇÃO 4
1.1 Motivação 4
1.2 Objetivos 4
1.3 Justificativa 4
1.4 Organização do Trabalho 5

2 ASPECTOS CONCEITUAIS 6
2.1 Oblivious Transfer (OT) 6
2.1.1 Definição 6
2.2 Variantes de OT 6
2.2.1 1-out-of-2 OT 6
2.2.2 1-out-of-n OT 7
2.2.3 k-out-of-n OT 7
2.2.4 Generalized Oblivious Transfer (GOT) 7
2.2.5 Definição 7
2.2.6 Funcionalidade 7
2.2.7 Relação com OT Clássico 8
2.2.8 Interpretação 8
2.2.9 Implementação 8
2.3 OT Extension 8
2.4 Criptossistema de Rabin 9
2.4.1 Definição 9

3 MÉTODO DO TRABALHO 11
3.1 Visão Geral do Processo 11
3.2 Fase 1 — Fundamentação Teórica 11
3.3 Fase 2 — Especificação de Requisitos 11
3.4 Fase 3 — Projeto da Solução 11
3.5 Fase 4 — Implementação Incremental 12
3.6 Fase 5 — Testes e Avaliação 12
3.7 Fase 6 — Consolidação e Análise Final 12

4 ESPECIFICAÇÃO DE REQUISITOS E SISTEMA 13
4.1 Requisitos Funcionais 13
4.2 Requisitos Não Funcionais 15
4.3 Modelo de Ameaça 15

4.4 Projeto Proposto . . . 17
4.5 Casos de uso e diagrama de fluxo . . . 17

5 PROJETO DA SOLUÇÃO . . . 21
5.1 Decisões de projeto . . . 21
5.2 Alternativas descartadas . . . 22
5.2.1 Biblioteca de OT . . . 22
5.2.2 Execução do OT . . . 24
5.2.3 Onde executa o receptor do OT . . . 24
5.2.4 Tecnologias do sistema web . . . 26
5.3 Estado da implementação . . . 26
5.3.1 O que os passos concluídos entregam . . . 27
5.3.2 Testes automatizados . . . 27
5.3.3 Medições . . . 28
5.3.4 O que ainda não existe . . . 28
5.4 Questões em aberto . . . 28

REFERÊNCIAS . . . 30

1 Introdução

Este documento apresenta o projeto de formatura intitulado “Aplicação de Protocolos de Oblivious Transfer para Preservação de Privacidade em Transações Digitais”.

1.1 Motivação

No cenário atual do comércio eletrônico, a coleta massiva de dados se tornou uma prática recorrente, de forma que qualquer compra torna a navegação pela internet uma experiência infestada de anúncios de produtos relacionados. Embora a vantagem de marketing dessa prática seja inegável, fornecer para os consumidores a privacidade que a maioria das lojas virtuais atualmente viola é um aspecto de grande interesse e necessidade para a sociedade.

1.2 Objetivos

Objetivo Geral: Desenvolver um protótipo funcional de um sistema de venda de ativos digitais em formato de catálogo, utilizando criptografia baseada em Oblivious Transfer (Transferência Inconsciente) para passar chaves de criptografia simétrica usadas para descriptografar os bens digitais adquiridos de Rede de Distribuição de Conteúdo (Content Delivery Network – CDN), garantindo que o servidor da loja seja incapaz de identificar quais produtos foram acessados ou adquiridos por um comprador específico.

Objetivos Específicos:

- Modelar a arquitetura do sistema para permitir a transação segura onde a escolha do cliente é ofuscada para o servidor.
- Implementar o algoritmo criptográfico escolhido (como 1-out-of-N) em uma solução com 3 componentes: site online, rede de distribuição de conteúdo (CDN) e programa local instalado no computador do cliente.
- Desenvolver mecanismos para validar a legitimidade da transação sem revelar o conteúdo da compra.
- Realizar testes de segurança e desempenho para validar a impossibilidade de rastreamento de preferências.

1.3 Justificativa

O projeto propõe uma solução técnica para um problema crescente de privacidade digital. O sistema permitirá que o comprador adquira produtos digitais dentre várias opções,

tendo acesso a apenas uma, mas de forma que o vendedor permaneça matematicamente incapaz de saber qual item específico foi selecionado pelo cliente.

A metodologia envolverá a implementação de algoritmos de Oblivious Transfer integrados a uma interface de venda, garantindo que a loja receba o pagamento, mas não consiga registrar históricos de compras, preferências ou interesses do usuário, assegurando assim um elevado nível de privacidade para os consumidores.

1.4 Organização do Trabalho

Este documento está estruturado da seguinte forma: o Capítulo 2 apresenta os conceitos fundamentais que embasam este projeto. O Capítulo 3 descreve a metodologia do trabalho. O Capítulo 4 detalha a especificação de requisitos do sistema. O Capítulo 5 expõe o projeto e o desenvolvimento da solução. Por fim, o Capítulo 6 reúne as considerações finais do estudo.

2 Aspectos Conceituais

2.1 Oblivious Transfer (OT)

2.1.1 Definição

Oblivious Transfer (OT) é um protocolo criptográfico entre duas partes:

- *Sender* S, que possui um conjunto de mensagens $X = \{x_0, x_1, \ldots, x_{n-1}\}$;
- *Receiver* R, que escolhe um índice $i \in \{0, \ldots, n-1\}$.

O protocolo garante que:

1. R aprende apenas x_i;
2. R não aprende nenhuma informação sobre x_j para $j \neq i$;
3. S não aprende qual índice i foi escolhido.

Formalmente, o objetivo é realizar:

$$(x_0, \ldots, x_{n-1}),\ i \longrightarrow x_i$$

sem violar as propriedades de privacidade.

2.2 Variantes de OT

2.2.1 1-out-of-2 OT

O caso mais simples ocorre quando o *sender* possui duas mensagens:

$$(x_0, x_1)$$

O *receiver* escolhe um bit:

$$r \in \{0, 1\}$$

Ao final do protocolo, o *receiver* obtém:

$$x_r$$

sem aprender x_{1-r}, e o *sender* não descobre o valor de r.

2.2.2 1-out-of-n OT

Generalização do caso anterior:

- *Sender* possui n mensagens;
- *Receiver* escolhe um índice $i \in \{0, \ldots, n-1\}$;

O *receiver* obtém apenas x_i, mantendo as mesmas garantias de privacidade.

2.2.3 k-out-of-n OT

Nesta variante:

- *Sender* possui n mensagens;
- *Receiver* escolhe um subconjunto $I \subseteq \{0, \ldots, n-1\}$ tal que $|I| = k$.

O *receiver* aprende:

$$\{x_i : i \in I\}$$

sem obter informação sobre as demais mensagens.

2.2.4 Generalized Oblivious Transfer (GOT)

O Generalized Oblivious Transfer (GOT) é uma generalização do Oblivious Transfer tradicional, introduzida por Ishai e Kushilevitz. (TASSA, 2011)

2.2.5 Definição

Seja Alice portadora de um conjunto de mensagens

$$M = \{M_1, M_2, \ldots, M_n\}.$$

Seja $[n] = \{1, 2, \ldots, n\}$ o conjunto de índices.

Define-se uma coleção $\mathcal{A} \subseteq 2^{[n]}$ de subconjuntos permitidos, chamada de coleção monotônica decrescente, isto é,

$$S \in \mathcal{A} \text{ e } S' \subseteq S \implies S' \in \mathcal{A}.$$

2.2.6 Funcionalidade

Bob escolhe um subconjunto $S \subseteq [n]$ tal que $S \in \mathcal{A}$. Ao final do protocolo:

- Bob aprende exatamente as mensagens $\{M_i\}_{i \in S}$;
- Bob não aprende nenhuma informação sobre $\{M_i\}_{i \notin S}$;
- Alice não aprende nenhuma informação sobre o subconjunto S escolhido por Bob.

2.2.7 Relação com OT Clássico

- No 1-out-of-n OT:
$$\mathcal{A} = \{S \subseteq [n] \mid |S| = 1\}.$$
- No k-out-of-n OT:
$$\mathcal{A} = \{S \subseteq [n] \mid |S| = k\}.$$
- No GOT:
$$\mathcal{A} \subseteq 2^{[n]}$$
pode ser qualquer coleção monotônica decrescente.

2.2.8 Interpretação

O GOT permite especificar políticas arbitrárias de acesso, desde que sejam fechadas por inclusão (monotonicidade decrescente). Por exemplo:

- todos os subconjuntos de tamanho $\leq k$;
- qualquer subconjunto que não contenha simultaneamente dois índices específicos;
- qualquer subconjunto contido em um conjunto autorizado T.

2.2.9 Implementação

Ishai e Kushilevitz mostraram que o GOT pode ser implementado a partir de múltiplas execuções paralelas de protocolos 1-out-of-2 OT.

2.3 OT Extension

Executar muitos OTs base (baseados em criptografia de chave pública) é computacionalmente custoso.

A técnica de OT Extension permite:

1. Executar um pequeno número de OTs base;
2. Expandir para milhares ou milhões de OTs usando apenas criptografia simétrica.

Isso reduz drasticamente o custo computacional, tornando MPC prático.

2.4 Criptossistema de Rabin

O criptossistema de Rabin é uma família de esquemas de criptografia de chave pública baseada em uma função armadilha cuja segurança, assim como a do RSA, está relacionada à dificuldade da fatoração de inteiros.

A função armadilha de Rabin tem a vantagem de que inverter essa função foi matematicamente provado ser tão difícil quanto fatorar inteiros, enquanto não há prova semelhante conhecida para a função armadilha do RSA. Ela tem a desvantagem de que cada saída da função de Rabin pode ser gerada por qualquer uma de quatro entradas possíveis; se cada saída for um texto cifrado, é necessária complexidade adicional na decriptação para identificar qual das quatro entradas possíveis era o texto plano verdadeiro. Tentativas ingênuas de contornar esse problema frequentemente possibilitam um ataque de texto cifrado escolhido para recuperar a chave secreta ou, ao codificar redundância no espaço do texto plano, invalidam a prova de segurança em relação à fatoração.

Esquemas de criptografia de chave pública baseados na função armadilha de Rabin são usados principalmente como exemplos em livros-texto. Em contraste, o RSA é a base de esquemas padrão de criptografia de chave pública, como o RSAES-PKCS1-v1_5 e o RSAES-OAEP, que são amplamente utilizados na prática. (RABIN, 2005)

2.4.1 Definição

Sejam p e q primos grandes, escolhidos de forma que

$$p \equiv q \equiv 3 \pmod 4,$$

e:

$$n = p \cdot q$$

Uma mensagem M é convertida em um número:

$$m < n$$

A encriptação é definida como:

$$c = m^2 \bmod n$$

O *ciphertext* é c.

A mensagem m pode ser recuperada a partir do texto cifrado c calculando sua raiz quadrada módulo n, como segue.

Calcule a raiz quadrada de c módulo p e módulo q utilizando as seguintes fórmulas:

$$m_p = c^{\frac{1}{4}(p+1)} \bmod p$$
$$m_q = c^{\frac{1}{4}(q+1)} \bmod q$$

Essas fórmulas só são válidas porque $p \equiv q \equiv 3 \pmod 4$: nesse caso $\frac{1}{4}(p+1)$ e $\frac{1}{4}(q+1)$ são inteiros e, pelo critério de Euler, $m_p^2 \equiv c \pmod p$ e $m_q^2 \equiv c \pmod q$. Por isso a geração de chaves deve impor essa condição.

Utilize o algoritmo de Euclides estendido para encontrar y_p e y_q tais que:

$$y_p \cdot p + y_q \cdot q = 1.$$

Aplique o Teorema Chinês do Resto para encontrar as quatro raízes quadradas de c módulo n:

$$\begin{aligned} r_1 &= (y_p \cdot p \cdot m_q + y_q \cdot q \cdot m_p) \bmod n \\ r_2 &= n - r_1 \\ r_3 &= (y_p \cdot p \cdot m_q - y_q \cdot q \cdot m_p) \bmod n \\ r_4 &= n - r_3 \end{aligned}$$

Um desses quatro valores é o texto plano original m, embora não seja possível determinar qual deles é o correto sem informação adicional.

3 Método do Trabalho

Este capítulo descreve o processo adotado para desenvolver o projeto, cujo objetivo é implementar um protótipo de livraria digital com preservação de privacidade por meio de Oblivious Transfer (OT). O método foi estruturado em fases de forma a manter alinhamento com os objetivos definidos no Capítulo 1.

3.1 Visão Geral do Processo

O processo de desenvolvimento foi organizado em seis fases principais: fundamentação teórica, especificação de requisitos, projeto da solução, implementação do protótipo, testes e avaliação, e consolidação dos resultados.

3.2 Fase 1 — Fundamentação Teórica

Na primeira fase, foram estudados os conceitos criptográficos necessários para o projeto, com ênfase em OT e suas variantes (como 1-out-of-n e GOT), além de técnicas relacionadas, como OT Extension e fundamentos do criptossistema de Rabin. Essa etapa forneceu base para as decisões arquiteturais e para a definição dos limites de segurança do protótipo (RABIN, 2005; TASSA, 2011).

3.3 Fase 2 — Especificação de Requisitos

Com base nos objetivos do trabalho, foi conduzida a elicitação e organização dos requisitos funcionais e não funcionais do sistema. Essa fase formaliza o que a solução deve fazer e quais restrições de desempenho, segurança e usabilidade devem ser atendidas. O detalhamento dessa etapa é apresentado no Capítulo 4.

3.4 Fase 3 — Projeto da Solução

Após a definição dos requisitos, pretende-se fazer o projeto com uma arquitetura inicial de referência. Os objetivos iniciais do projeto são: organização em três componentes (sistema web, CDN e programa local), modelagem preliminar do fluxo de compra privada, alternativas de agrupamento de itens por faixa de preço e possíveis formatos de artefatos de criptografia enviados. Nessa etapa, os critérios de separação entre as operações do sistema web e as operações locais do cliente serão definidos conforme os resultados parciais. O detalhamento dessa etapa é apresentado no Capítulo 5.

3.5 Fase 4 — Implementação Incremental

A implementação deve ser conduzida em incrementos. A sequência abaixo representa uma proposta inicial, passível de ajustes durante o desenvolvimento:

- estrutura de catálogo e cadastro de itens;
- fluxo de compra com coleta mínima de dados do cliente;
- execução do mecanismo criptográfico associado à escolha privada;
- emissão de tokens com assinatura cega e entrega do conteúdo cifrado pela CDN;
- suporte ao processo de recuperação local dos itens adquiridos.

Esse encadeamento foi definido como estratégia de partida e poderá ser reordenado caso surjam problemas técnicos ou necessidades de validação não previstas. O detalhamento dessa etapa é apresentado no Capítulo 5.

3.6 Fase 5 — Testes e Avaliação

Na fase de avaliação, pretende-se priorizar três questões: (i) eficácia do sistema, com foco no atendimento aos requisitos; (ii) nível de privacidade, buscando provas de que o servidor não infere a escolha específica do cliente; e (iii) desempenho, com medições de tempo de busca, compra e entrega. Os procedimentos exatos de teste e o recorte final de métricas serão definidos ao longo da implementação e posteriormente descritos no Capítulo 5.

3.7 Fase 6 — Consolidação e Análise Final

Na etapa final, pretende-se consolidar os resultados alcançados e analisá-los com os objetivos do projeto, dando ênfase nas limitações observadas, contribuições efetivas e possibilidades de continuidade. O nível de profundidade dessa análise dependerá do que for possível concluir nas etapas anteriores e será apresentado no Capítulo 6.

4 Especificação de Requisitos e Sistema

Este capítulo apresenta os requisitos da solução proposta para uma livraria digital em modelo de catálogo, que utiliza um algoritmo *oblivious* para viabilizar a venda dos itens mantendo a privacidade sobre qual item foi comprado.

4.1 Requisitos Funcionais

- **RF01 — Cadastro de administradores:** o sistema deve permitir o cadastro, autenticação e gerenciamento de perfis de administradores.
- **RF02 — Manuseio de livros no catálogo:** o sistema deve permitir ao administrador cadastrar, editar, consultar e remover livros do catálogo.
- **RF03 — Política de preços discretos:** o sistema deve permitir a precificação de cada livro dentro de um conjunto de preços pré-definidos.
- **RF04 — Busca e navegação no catálogo:** o sistema deve permitir que clientes pesquisem livros por critérios como título, autor, categoria e preço. O catálogo é publicado pelo sistema web como um pacote completo, na mesma forma para todos os clientes, e a busca e a navegação são feitas no programa local, de modo que o servidor não observe os critérios de busca nem os livros consultados. O sistema web não oferece vitrine pública nem imagens de capa avulsas; sua única página pública é a de apresentação e download, igual para todos os visitantes.
- **RF05 — Compra sem cadastro de cliente:** o sistema deve permitir compra sem criação de conta de cliente.
- **RF06 — E-mail obrigatório na compra:** no fluxo de compra, o cliente deve informar um e-mail válido para recebimento da resposta do protocolo de OT, a partir da qual o programa local extrai a chave do produto digital.
- **RF07 — Compra privada de N itens em grupo:** o cliente deve conseguir comprar N livros dentre um grupo maior de livros com o mesmo preço sem que a informação dos itens específicos comprados seja enviada ao servidor em formato que ele consiga decifrar. Cada grupo possui uma versão imutável (composição e ordem dos livros), e a compra referencia essa versão.
- **RF08 — Entrega de material criptográfico para cliente:** o sistema deve fornecer ao cliente os dados necessários (a resposta do protocolo de OT e informações

relacionadas) para que as chaves simétricas dos livros escolhidos sejam extraídas em ambiente local. O servidor não conhece as chaves finais obtidas pelo cliente.

- **RF09 — Arquitetura com três componentes:** a solução deve ser composta por (i) um sistema web para administração e publicação do catálogo, pagamento, execução do OT e assinatura cega de tokens; (ii) uma CDN, que armazena os livros cifrados e os entrega mediante token válido; e (iii) um programa local, instalado no computador do cliente e não fornecido pelo sistema web em tempo de execução, que permite navegar no catálogo, guarda o estado secreto do cliente, comunica-se apenas com o sistema web e com a CDN e descriptografa os livros.
- **RF10 — Descriptografia local:** o cliente deve conseguir descriptografar o item resgatado por meio do programa local, a partir do livro cifrado e da resposta do OT. A chave do livro é obtida e usada apenas no computador do cliente e nunca é transmitida pela rede. O programa local comunica-se exclusivamente com o sistema web e com a CDN, sem telemetria nem serviços de terceiros.
- **RF11 — Gestão do segredo no programa local:** o programa local deve gerar a requisição de OT a partir de uma semente secreta, obtida de um gerador criptograficamente seguro do sistema, com pelo menos 128 bits, nova a cada compra e nunca reutilizada. O programa guarda a semente e as escolhas, necessárias para extrair, a partir da resposta do servidor, as chaves criptográficas dos itens adquiridos, inclusive a partir da resposta recebida por e-mail (RF14) caso a compra seja interrompida após o pagamento. A perda da semente implica a perda de acesso aos itens daquela compra.
- **RF12 — Não acesso a itens não comprados:** o cliente não deve conseguir descriptografar livros que não tenham sido comprados.
- **RF13 — Processamento de pagamento:** o sistema deve realizar o processamento do pagamento da compra por meio de mecanismo integrado de cobrança (gateway de pagamento) e retornar o status da transação ao cliente.
- **RF14 — Envio da resposta do OT por e-mail:** após a confirmação do pagamento, o sistema deve enviar para o e-mail informado a resposta do OT correspondente à compra. O e-mail não contém a chave do livro, que o servidor desconhece.
- **RF15 — Persistência para reenvio ao cliente:** o sistema deve manter os dados da compra de forma que o cliente possa solicitar o reenvio da resposta do OT para o mesmo e-mail informado originalmente, dentro de um prazo definido pela plataforma.
- **RF16 — Emissão de tokens com assinatura cega:** para cada compra de N itens, o sistema web deve assinar às cegas N valores gerados pelo cliente, resultando

em N tokens que autorizam, cada um, o download de um livro na CDN. Os tokens não devem poder ser ligados à sessão de assinatura.

- **RF17 — Entrega de livros pela CDN mediante token:** a CDN deve entregar um livro cifrado apenas mediante a apresentação de um token com assinatura válida e ainda não utilizado.
- **RF18 — Reporte de uso da CDN:** a CDN deve reportar ao sistema web os tokens utilizados e a quantidade de livros entregues, para fins de remuneração. O reporte não deve conter o livro associado a cada token nem horários que permitam correlação com as compras.

4.2 Requisitos Não Funcionais

- **RNF01 — Segurança criptográfica:** o mecanismo de proteção deve atingir um nível mínimo de 128 bits de segurança, isto é, um ataque ao mecanismo deve exigir da ordem de 2^{128} operações.
- **RNF02 — Tempo limite de busca:** cada operação de busca no catálogo deve ser concluída em um tempo de resposta limite de 5 segundos.
- **RNF03 — Tempo limite de finalização da compra:** a finalização da compra deve ser concluída em um tempo de resposta limite de 1 minuto.
- **RNF04 — Tempo limite de envio por e-mail:** o sistema deve enviar o e-mail contendo a resposta do OT dentro de um tempo de resposta limite de 1 minuto após confirmação do pagamento.
- **RNF05 — Capacidade de processamento:** a plataforma deve suportar uma quantidade mínima de 600 transações por minuto.
- **RNF06 — Disponibilidade:** a plataforma deve atingir um valor mínimo de 99% de *uptime* por mês.
- **RNF07 — Usabilidade:** o fluxo de compra deve atingir uma taxa mínima de conclusão sem suporte externo igual a 99%.
- **RNF08 — Manutenibilidade:** alterações evolutivas de baixa complexidade devem ser implementadas em um tempo máximo de 8 horas úteis de manutenção.

4.3 Modelo de Ameaça

O projeto adota as seguintes premissas de segurança, que orientam as decisões de arquitetura:

- **Site e CDN não são coniventes.** Essa premissa é garantida por auditoria. Com isso, o site sabe *quem* comprou, mas não *o quê*; a CDN sabe *o quê* foi baixado, mas não *quem* comprou. Essa separação de conhecimento é intencional e não constitui uma falha do sistema.

- **O reporte da CDN ao site deve ser mínimo.** Ele contém apenas os tokens utilizados e a quantidade de entregas, nunca o livro associado a cada token nem horários que permitam correlacionar downloads com compras (RF18).

- **Os tokens não podem ser ligados à compra.** A assinatura cega garante que o site, ao receber um token no reporte da CDN, não consiga associá-lo à sessão de assinatura em que foi emitido (RF16).

- **O servidor não conhece a chave final do cliente.** O site executa sua parte do OT sem saber quais chaves o cliente extraiu. Por isso o e-mail (RF14) e o reenvio (RF15) carregam a *resposta do OT*, e não a chave. Somente o programa local, que guarda o segredo da requisição (RF11), consegue extrair a chave.

- **O código do receptor do OT não vem do servidor.** A requisição de OT é gerada pelo programa local, instalado pelo cliente, e não por código entregue pelo sistema web durante a compra. Assim, a privacidade da escolha (RF07) decorre do protocolo de OT e não depende de o servidor fornecer código honesto.

- **Navegação e busca não revelam interesse.** O programa local baixa o catálogo inteiro, com miniaturas de capa de tamanho fixo, e faz a busca e a navegação localmente (RF04); o sistema web observa apenas que o pacote completo foi baixado. Como o código de navegação não é fornecido pelo sistema web, a proteção não depende de o servidor entregar código honesto. Trata-se de uma medida de engenharia, não de uma garantia do protocolo; complementam-na a ausência de vitrine pública, de imagens avulsas e de registros de acesso no sistema web.

- **O programa local fala apenas com o sistema web e com a CDN.** A chave final e a semente nunca deixam o computador do cliente (RF10, RF11). O endereço de rede do cliente é visível à CDN; a ligação entre esse dado e a identidade do comprador exigiria conivência entre site e CDN, excluída pela premissa acima.

- **O cegamento dos tokens é removido no programa local.** Os valores cegados são gerados e descegados no programa local, e não no navegador, pelo mesmo motivo do item sobre o código do receptor.

4.4 Projeto Proposto

A partir dos requisitos, a ideia inicial do projeto é uma plataforma de venda de chaves de livros criptografados, que são distribuídos por uma CDN. A solução é composta por três componentes:

1. **Sistema web (site):** administração e publicação do catálogo, preços discretos, pagamento via gateway, execução do OT e assinatura cega de tokens. A compra não exige conta; apenas o e-mail é solicitado.
2. **CDN:** armazena os livros cifrados e entrega um livro mediante um token com assinatura cega válido. Posteriormente, reporta ao site os tokens utilizados para ser remunerada.
3. **Programa local:** instalado no computador do cliente, baixa o catálogo completo e permite buscar e navegar localmente, gera a requisição de OT e guarda seu segredo, comunica-se com o sistema web e com a CDN, remove o cegamento dos tokens e descriptografa os livros localmente.

No fluxo de compra, o cliente escolhe um grupo de livros de mesmo preço e obtém, por meio do OT, as chaves de N itens desse grupo, sem que o site saiba quais foram escolhidos. O cliente recebe também N tokens com assinatura cega, com os quais baixa os livros cifrados na CDN, e em seguida os descriptografa localmente. Como cada download consome um token e a CDN reporta os tokens utilizados, torna-se possível a remuneração justa da CDN sem revelar ao site quais livros foram adquiridos.

A execução do OT é síncrona: durante a compra, o programa local envia a requisição de OT e recebe a resposta em uma única troca de mensagens com o sistema web, dentro do limite de RNF03. Toda a aleatoriedade do lado receptor é derivada de uma semente guardada pelo programa local (RF11); por isso, se a compra for interrompida após o pagamento, o programa reproduz o estado do receptor a partir da semente e extrai as chaves usando a resposta do OT recebida por e-mail (RF14). Esse desenho exige um protocolo de OT em que o sistema web responda em uma única mensagem e em que o estado do receptor seja reproduzível a partir da semente.

Os livros de um grupo de mesmo preço são organizados em versões imutáveis: a inclusão, remoção ou alteração de preço de um livro gera uma nova versão do grupo, e cada compra referencia a versão sobre a qual os índices do OT foram escolhidos.

4.5 Casos de uso e diagrama de fluxo

A Figura 1 apresenta o fluxo da administração do catálogo até a compra, e a Figura 2 apresenta a entrega pela CDN, o reporte de uso e a descriptografia local, incluindo a retomada de uma compra interrompida após o pagamento.

Tabela 1 – Agentes e Casos de Uso do Sistema

ID	**Agente**	**Caso de Uso**
UC01	Administrador	Gerenciar conta de administrador (cadastro, login e edição)
UC02	Administrador	Gerenciar catálogo de livros (cadastrar, editar, consultar e remover)
UC03	Administrador	Definir política de preços discretos para os livros
UC04	Cliente (Programa Local)	Buscar e navegar no catálogo de livros
UC05	Cliente	Realizar compra sem necessidade de cadastro
UC06	Cliente	Informar e-mail válido para recebimento da resposta do OT
UC07	Cliente	Realizar compra privada de N itens utilizando protocolo *oblivious transfer*
UC08	Sistema	Processar pagamento da compra
UC09	Sistema	Executar o OT e entregar a resposta ao cliente
UC10	Sistema	Enviar resposta do OT por e-mail após pagamento
UC11	Cliente	Solicitar reenvio da resposta do OT
UC12	Sistema	Persistir dados da compra para reenvio
UC13	Sistema	Assinar às cegas os N valores do cliente (emissão de tokens)
UC14	Cliente	Baixar livro cifrado na CDN apresentando um token (via programa local)
UC15	CDN	Validar token e entregar livro cifrado
UC16	CDN	Reportar tokens utilizados e quantidade ao site
UC17	Sistema	Validar tokens reportados e remunerar a CDN
UC18	Cliente (Programa Local)	Descriptografar conteúdo localmente
UC19	Sistema (Programa Local)	Guardar o segredo da requisição e extrair a chave localmente
UC20	Sistema	Garantir que itens não comprados não possam ser acessados

Figura 1 – Diagrama de sequência: administração, navegação e compra

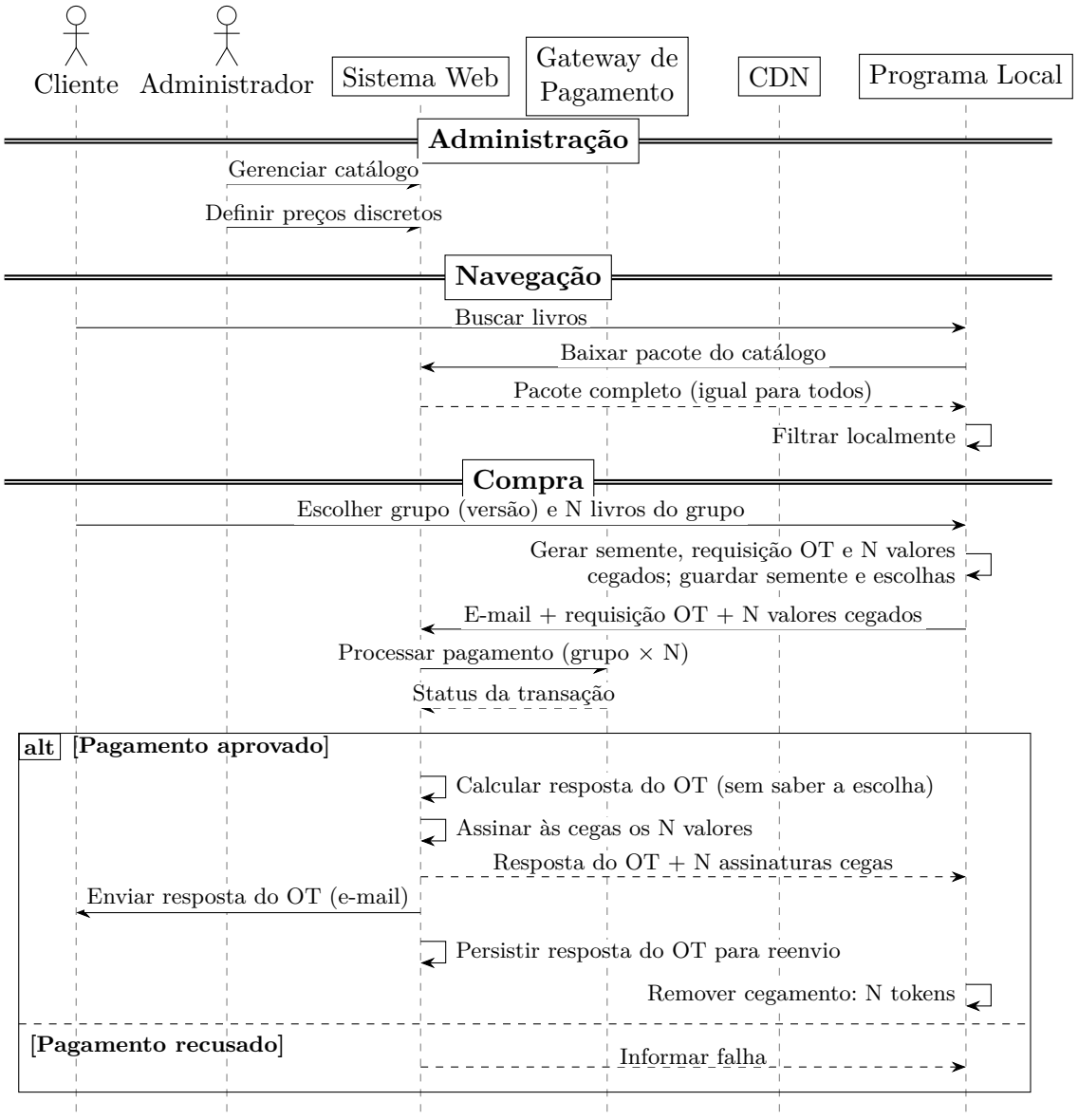
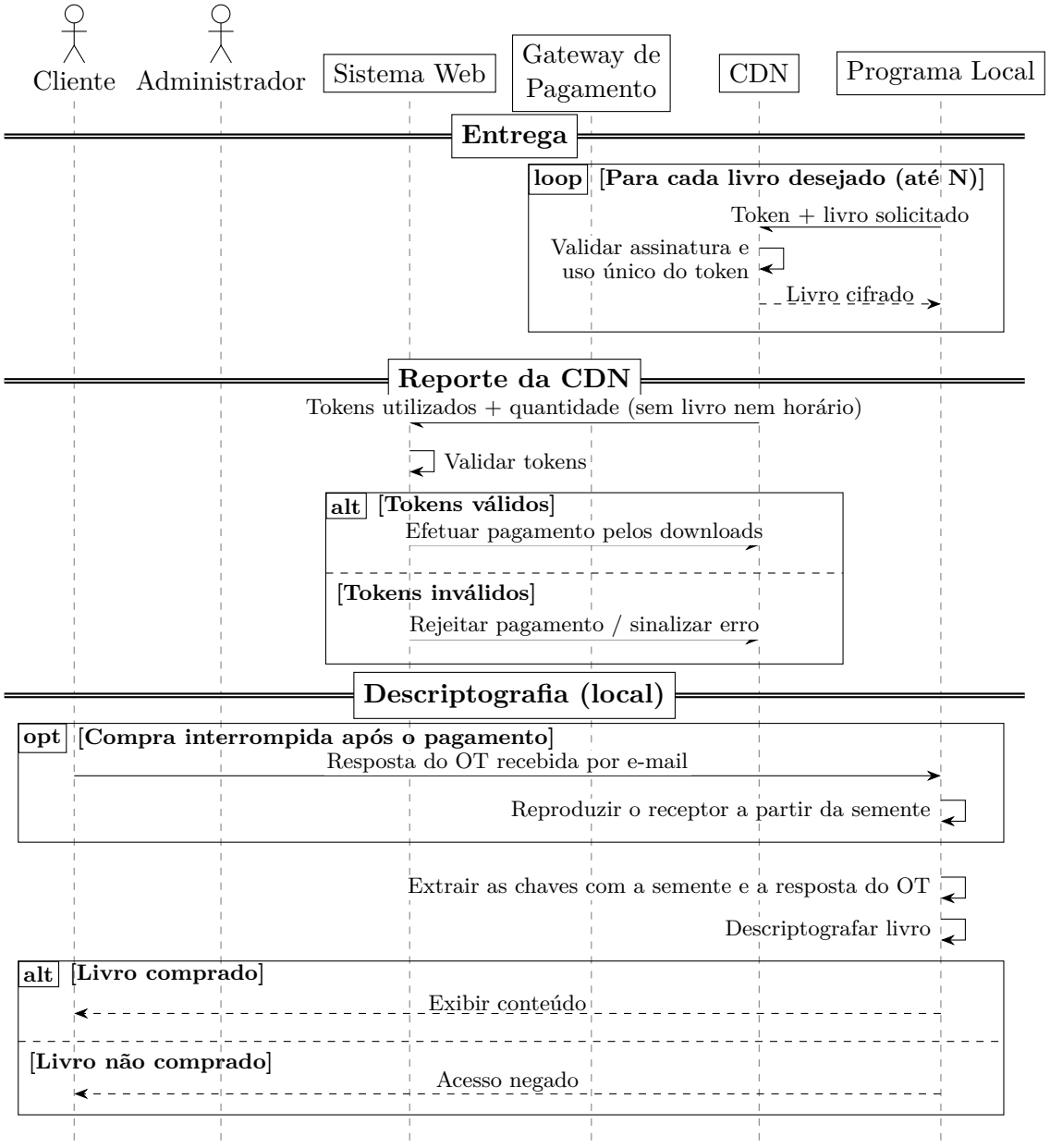


Figura 2 – Diagrama de sequência: entrega, reporte da CDN e descriptografia local



5 Projeto da Solução

Este capítulo registra as decisões de projeto tomadas até o momento e as questões que permanecem em aberto. Ele será expandido com o detalhamento da implementação e dos resultados das fases 4 e 5.

5.1 Decisões de projeto

- **Execução do OT:** síncrona na compra, com o estado do receptor reproduzível a partir de uma semente guardada pelo programa local (RF11, RF14).
- **Biblioteca de OT:** libOTe (RINDAL, 2026), com o protocolo de Masny e Rindal (2019) como OT base 1-out-of-2. A compilação no ambiente do cliente e a reprodutibilidade do receptor a partir da semente são verificadas por testes automatizados durante a implementação.
- **Sistema web:** linguagem Go, banco PostgreSQL, páginas de administração renderizadas no servidor, sem recursos de terceiros; catálogo publicado como pacote completo, sem vitrine pública; biblioteca de OT acessada por um binário auxiliar. A única página pública é a de apresentação e download, sem cookies, scripts ou recursos de terceiros, que exibe a versão, o resumo SHA-256 e a chave pública do catálogo atual para que qualquer pessoa possa conferi-los.
- **Navegação e busca:** no programa local, sobre o pacote completo do catálogo. As capas seguem no pacote como miniaturas de 200 por 300 pixels em JPEG, geradas no envio da capa, para que o tamanho do pacote não dependa das imagens originais.
- **Pacote do catálogo:** um arquivo ZIP único por versão, com os metadados em JSON e as miniaturas. O arquivo é gerado de forma determinística, de modo que o mesmo conteúdo produz sempre os mesmos bytes e o resumo SHA-256 identifica a versão. O programa local sempre baixa o pacote completo, revalidando-o pelo resumo, para que todos os clientes façam a mesma requisição. O sistema web assina cada versão com Ed25519 e publica versão, resumo e assinatura, o que permite a qualquer pessoa verificar que recebeu o mesmo catálogo que os demais. O programa local confere a assinatura com a chave pública embutida e recusa versões anteriores à que já possui.
- **Assinatura cega:** RSA cego padronizado (DENIS; JACOBS; WOOD, 2023).
- **Programa local:** conecta-se apenas ao sistema web e à CDN.

5.2 Alternativas descartadas

Esta seção compara, para cada decisão de projeto, a alternativa escolhida com as descartadas, usando os mesmos critérios para todas. A comparação é qualitativa: registra propriedades verificadas no código-fonte ou na documentação de cada alternativa e marca como "não verificado" o que não foi confirmado.

5.2.1 Biblioteca de OT

Critérios: (C1) compatibilidade com o fluxo de compra, em que o programa local envia a requisição e o sistema web responde em uma única mensagem; (C2) possibilidade de guardar ou reproduzir o estado do receptor entre a requisição e a extração das chaves (RF11, RF14); (C3) execução no Windows, ambiente do programa local; (C4) maturidade e recomendação de uso pelos autores; (C5) aderência ao foco do trabalho, que é o uso de protocolos de OT.

Tabela 2 – Comparativo das bibliotecas de OT avaliadas

Alternativa	**C1 – Mensagens**	**C2 – Estado do receptor**	**C3 – Windows**	**C4 – Maturidade**	**Resultado**
libOTe, OT base de Masny e Rindal (2019)	Compatível: a requisição do receptor não depende da mensagem do *sender*, que segue na resposta	Em memória, mas reproduzível: toda a aleatoriedade do receptor vem do gerador recebido como parâmetro (verificado no código)	Sim, segundo os autores	Acadêmica, licença MIT, amplamente citada	**Escolhida**
emp-ot, protocolo PVW	Compatível: receptor envia primeiro, duas mensagens	Em memória, chamada bloqueante; reprodutibilidade não verificada	**Não** (apenas Linux e macOS)	Software de pesquisa, sem auditoria independente, segundo os autores	Descartada (C3)
circl, Simplest OT	Exige uma troca adicional: a requisição do receptor depende da mensagem inicial do *sender*	API dividida em rodadas, favorável	Sim	Apenas 1-out-of-2; pacote fora da versão mais recente do módulo	Descartada (C1, C4)
mpz, Chou–Orlandi	Mesma dependência da mensagem inicial do *sender* (protocolo Chou–Orlandi)	Favorável: máquinas de estado sem rede, com mensagens explícitas	Não verificado	Autores recomendam não usar em produção; mudanças incompatíveis frequentes	Descartada (C4)
swanky (Naor–Pinkas, Chou–Orlandi)	Não verificado	Não verificado	Não verificado	Autores recomendam não usar em produção	Descartada (C4)
OPRF (RFC 9497) usado para construir 1-out-of-n	Compatível: cliente envia primeiro, duas mensagens	API em passos, favorável	Sim	Padronizado, várias implementações independentes	Descartada (C5)

A alternativa baseada em OPRF é tecnicamente adequada, mas não é um protocolo de OT: ela obtém a funcionalidade de 1-out-of-n a partir de uma função pseudoaleatória avaliada obliviamente, o que deslocaria o foco do trabalho.

5.2.2 Execução do OT

Critérios: utilidade do envio da resposta por e-mail (RF14) e do reenvio (RF15); recuperação de uma compra interrompida após o pagamento (RNF07); quantidade de código criptográfico próprio.

Tabela 3 – Comparativo das formas de execução do OT

Alternativa	**RF14 e RF15**	**Compra interrompida**	**Código criptográfico próprio**	**Resultado**
Síncrona, com semente guardada	Úteis: a resposta recebida por e-mail permite extrair as chaves	Recuperável a partir da semente	Nenhum além de inicializar o gerador com a semente; depende da premissa de determinismo do receptor	**Escolhida**
Síncrona, sem estado guardado	Perdem a utilidade: sem o segredo, a resposta não serve	Irrecuperável: o cliente paga e perde a compra	Nenhum	Descartada
Adaptador com estado serializado	Úteis	Recuperável a partir do estado salvo	Reorganização do protocolo em duas etapas sobre as primitivas da biblioteca	Descartada

5.2.3 Onde executa o receptor do OT

Critérios: natureza da privacidade da escolha (RF07), usabilidade, viabilidade técnica com a biblioteca escolhida e armazenamento da semente (RF11).

Tabela 4 – Comparativo dos locais de execução do receptor

Alternativa	**Privacidade da escolha**	**Usabilidade**	**Biblioteca de OT**	**Semente**	**Resultado**
Programa local com acesso restrito ao sistema web e à CDN	Garantia do protocolo	Sem transporte manual de arquivos	Execução nativa	Disco do cliente	**Escolhida**
Programa local sem rede (versão 2 desta especificação)	Garantia do protocolo	Cliente transporta arquivos entre navegador e programa	Execução nativa	Disco do cliente	Descartada
Navegador, com código fornecido pelo sistema web	Premissa: o servidor fornece código honesto	Nada a instalar	Exigiria compilação para WebAssembly (viabilidade não verificada)	Guardada no navegador, sujeita a remoção	Descartada
Navegador, com código de outra origem (extensão ou página estática auditável)	Premissa mais fraca: código auditável e independente do servidor	Instalação de extensão ou uso de página separada	Mesma limitação da alternativa anterior	Guardada no navegador	Descartada

5.2.4 Tecnologias do sistema web

Tabela 5 – Comparativo das tecnologias do sistema web

Tema	**Escolhida**	**Descartada**	**Motivo**
Linguagem	Go	TypeScript (Node.js); Python	Go oferece assinatura cega RSA padronizada (DENIS; JACOBS; WOOD, 2023) em biblioteca mantida, biblioteca padrão com servidor HTTP e criptografia, e binário único. Em Python não foi identificada implementação consolidada da RFC 9474.
Banco de dados	PostgreSQL	SQLite	Escritas concorrentes representativas para o teste de carga (RNF05).
Interface	HTML gerado no servidor	Aplicação de página única (SPA)	Menor superfície de ataque e nenhuma dependência de código de terceiros no navegador.
Busca (RF04)	Pacote completo, navegado no programa local	Vitrine no navegador com filtro local; busca no servidor sem registros	Na vitrine no navegador, o filtro é código fornecido pelo sistema web, e a proteção dependeria de ele ser honesto; na busca no servidor, o sistema web observa as consultas e a proteção dependeria apenas de disciplina operacional.
Acesso à biblioteca de OT	Binário auxiliar	Ligação direta ao código C++	Isola o código C++ e desacopla a compilação da biblioteca da compilação do sistema web.

5.3 Estado da implementação

Esta seção registra o andamento da Fase 4 em 26 de outubro de 2026. A implementação segue os passos planejados para o sistema web, na ordem do método; a CDN e o programa local ainda não foram iniciados. O código está no repositório do projeto (<https://github.com/migs39/tcc>).

Tabela 6 – Passos da implementação do sistema web

Passo	**Entrega**	**Requisitos**	**Situação**
0	Base: banco de dados, migrações e testes isolados	—	Concluído
1	Cadastro, autenticação e gestão de administradores	RF01, UC01	Concluído
2	Catálogo, preços discretos, capas e grupos versionados	RF02, RF03, RF07, UC02, UC03	Concluído
3	Pacote do catálogo e página pública	RF04, UC04	Concluído
4	Compra sem conta, e-mail e pagamento	RF05, RF06, RF13	Não iniciado
5	Envio da resposta do OT por e-mail	RF14, RNF04	Não iniciado
6	OT no sistema web, com a biblioteca libOTe	RF07, RF08	Não iniciado
7	Emissão de tokens com assinatura cega	RF16	Não iniciado
8	Recebimento e validação do reporte da CDN	RF18, UC17	Não iniciado
9	Reenvio da resposta do OT	RF15	Não iniciado
10	Testes de privacidade e de carga	RNF02–RNF05	Não iniciado

5.3.1 O que os passos concluídos entregam

- **Administradores (RF01):** cadastro, autenticação e gestão de perfis, inclusive desativação, que nunca deixa o sistema sem administrador ativo. As senhas são guardadas com argon2id; a sessão usa um token aleatório de 256 bits, do qual só o resumo fica no banco. Requisições de alteração vindas de outra origem são recusadas.

- **Catálogo e preços (RF02, RF03):** livros com título, autor, categoria, descrição, ano, ISBN, editora e capa; faixas de preço e categorias mantidas pelo administrador. A restrição de preço ao conjunto pré-definido é imposta pelo código e também pelo banco de dados, de modo que nenhum livro fica com preço fora do conjunto mesmo que o código seja contornado. Remover um livro o desativa, preservando as versões de grupo que o referenciam.

- **Grupos versionados (RF07):** cada mudança na composição de um grupo de mesmo preço gera uma versão imutável; editar dados de um livro sem mudar seu preço não gera versão. A posição do livro na versão é o índice usado pelo OT.

- **Pacote do catálogo (RF04):** o sistema web publica o catálogo como um arquivo ZIP determinístico, com os metadados dos livros ativos, as versões atuais dos grupos e as miniaturas das capas, assinado com Ed25519. A página pública de apresentação e download mostra a versão, o resumo e a chave pública do catálogo atual.

5.3.2 Testes automatizados

O sistema web tem 29 testes automatizados, executados contra um banco PostgreSQL real, cada teste em um esquema isolado. Os grupos de testes são:

- **Administradores (9 testes):** formato e verificação do resumo de senha; cadastro e validações; autenticação; desativação, que encerra sessões e bloqueia o acesso; troca de senha, que encerra as sessões; sessão guardada apenas como resumo.
- **Catálogo (9 testes):** preço fora do conjunto recusado pelo código e pelo banco (RF03); regras de faixas e categorias; validação de ano e ISBN; sequência e imutabilidade das versões de grupo; capas e miniaturas.
- **Pacote do catálogo (3 testes):** conteúdo e ordem dos grupos; determinismo; geração de versões; recusa de pacote adulterado, de número de versão trocado e de chave errada.
- **Interface web (8 testes):** acesso administrativo restrito a administradores autenticados; recusa de requisições de outra origem; fluxo completo do catálogo; revalidação do pacote; página pública sem cookies, scripts ou recursos de terceiros.

Parte desses testes já verifica propriedades de privacidade do que foi implementado: dois visitantes recebem exatamente o mesmo pacote do catálogo, o sistema web não oferece imagens de capa avulsas e a página pública não carrega recursos externos. Os testes de privacidade do fluxo de compra e os testes de desempenho (RNF02 a RNF05) dependem dos passos seguintes.

5.3.3 Medições

As miniaturas de capa, de 200 por 300 pixels em JPEG, mediram de 1,6 KB (capa de cor lisa) a 19,8 KB (pior caso, imagem de ruído aleatório). Com mil livros, o pacote do catálogo fica abaixo de cerca de 20 MB.

5.3.4 O que ainda não existe

Nenhuma parte criptográfica do fluxo de compra foi implementada: o OT, os tokens com assinatura cega e a descriptografia ainda não existem. A CDN (RF17, RF18) e o programa local (navegação do RF04, RF10 a RF12) também não foram iniciados. Os riscos assumidos na escolha da biblioteca de OT (compilação no Windows e reprodutibilidade do receptor a partir da semente) serão verificados por testes no passo 6.

5.4 Questões em aberto

- Variante de OT: 1-out-of-n repetido (por exemplo, a construção de Naor e Pinkas (1999) sobre 1-out-of-2), k-out-of-n ou GOT.
- Algoritmo simétrico para cifrar os livros, dentro de RNF01.

- Formato dos artefatos: requisição OT, resposta OT, tokens e livro cifrado.
- Tamanho mínimo de um grupo de mesmo preço.
- Política de chaves da assinatura cega e tamanho do conjunto de anonimato dos tokens.
- Token de uso único ou com novo download do mesmo livro.
- Frequência e lotes do reporte da CDN e remuneração por token.
- Prazo de reenvio da resposta do OT (RF15).
- Gateway de pagamento e sua integração com o programa local.
- Proteção da semente armazenada no programa local.
- Compromissos das chaves dos livros no pacote do catálogo, para o programa local conferir as chaves extraídas (defesa contra o ataque de falha seletiva no OT).
- Interface do programa local, que também passa a ser a vitrine do catálogo.

Referências

DENIS, F.; JACOBS, F.; WOOD, C. A. *RSA Blind Signatures*. 2023. RFC 9474, Internet Research Task Force (IRTF). Disponível em: <https://www.rfc-editor.org/rfc/rfc9474>.

MASNY, D.; RINDAL, P. Endemic oblivious transfer. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. [S.l.: s.n.], 2019. p. 309–326.

NAOR, M.; PINKAS, B. Oblivious transfer and polynomial evaluation. In: *Proceedings of the 31st Annual ACM Symposium on Theory of Computing (STOC)*. [S.l.: s.n.], 1999. p. 245–254.

RABIN, M. O. How to exchange secrets with oblivious transfer. *Cryptology ePrint Archive, Report 2005/187*, 2005. Disponível em: <https://eprint.iacr.org/2005/187.pdf>.

RINDAL, P. *libOTe: an efficient, portable, and easy to use Oblivious Transfer library*. 2026. Disponível em: <https://github.com/osu-crypto/libOTe>.

TASSA, T. Generalized oblivious transfer by secret sharing. *Designs, Codes and Cryptography*, 2011. Disponível em: <https://www.openu.ac.il/home/tamirtassa/Publications/got.pdf>.